

Schritt für Schritt: Dienstplan online (URL statt nur Mac)

Ziel: Die App läuft unter einer **https://...** -Adresse. Daten liegen in der **Cloud-Datenbank**. Login bleibt **passwortgeschützt**.

Du brauchst: E-Mail-Adresse für drei kostenlose Dienste (**Neon**, **GitHub**, **Vercel**), etwa **30–45 Minuten** beim ersten Mal.

Teil A — Datenbank in der Cloud (Neon)

1. Öffne **https://neon.tech** und registriere dich.
2. **Create project** (Projektname beliebig, Region z. B. Frankfurt wenn verfügbar).
3. Warte, bis das Projekt fertig ist. Kopiere den **Connection string** für Postgres (beginnt mit **postgresql://** oder **postgres://**).
4. **Nichts** davon in GitHub oder Screenshots posten — das ist ein Geheimnis.

Weiter mit Teil B auf deinem Mac.

Teil B — Einmalig: Datenbank füllen (auf deinem Mac, Terminal)

Alles im Ordner **web** (da liegt **package.json**).

1. Terminal öffnen:

```
cd "/Users/timi_1/Desktop/Dienstplan Tool/web"
```

2. Neon-String einsetzen (Zeile in **eine** Zeile, Anführungszeichen behalten):

```
export DATABASE_URL="HIER_DEN_NEON_CONNECTION_STRING_EINFÜGEN"
```

3. Zufallswert für die Session (einfach ausführen):

```
export SESSION_SECRET="$(openssl rand -base64 32)"
```

4. **Dein** Login-Passwort für den Planer (selbst wählen, merken):

```
export PLANNER_PASSWORD="DeinSicheresPasswort"
```

5. Schema anlegen und befüllen:

```
npx prisma db push  
npx prisma db seed
```

6. Wenn am Ende „Seed OK“ o. Ä. steht: Teil B ist fertig.

Login später auf der Website: E-Mail **planer@local**, Passwort = das, was du bei **PLANNER_PASSWORD** gesetzt hast.

Teil C — Code auf GitHub

1. Auf **https://github.com** einloggen → **Repositories** → **New** (neues Repository).
2. Name z. B. **dienstplan-app**, **ohne** README / .gitignore von GitHub anhaken (Repository **leer** lassen).
3. Auf dem Mac, noch im Ordner **web**:

```
git init  
git add .  
git commit -m "Initial commit: Dienstplan app"  
git branch -M main
```

4. Bei GitHub die beiden Zeilen „**push an existing repository**“ anzeigen lassen (HTTPS-URL kopieren), dann **deinen** GitHub-Benutzernamen und **Repo-Namen** einsetzen:

```
git remote add origin https://github.com/DEIN_USER/dienstplan-app.git  
git push -u origin main
```

5. Wenn der Push durch ist: Teil C ist fertig.

Teil D — App bei Vercel veröffentlichen

1. Öffne **https://vercel.com** und melde dich an (am einfachsten „**Continue with GitHub**“).
2. **Add New...** → **Project** → dein Repo **dienstplan-app** auswählen → **Import**.
3. **Wichtig — Umgebungsvariablen** (vor dem ersten Deploy oder unter **Settings** → **Environment Variables**):

Name	Wert
DATABASE_URL	Derselbe Neon-Connection-String wie in Teil B

Name	Wert
<code>SESSION_SECRET</code>	Derselbe Wert wie in Teil B (Terminal: <code>echo \$SESSION_SECRET</code> — vorher nicht Terminal schließen, oder neu mit <code>openssl rand -base64 32</code> erzeugen und überall gleich bei Vercel eintragen)
<code>PLANNER_PASSWORD</code>	Dasselbe Passwort wie in Teil B

4. **Framework Preset:** Next.js (sollte automatisch erkannt werden). **Root Directory:** leer lassen — nur nötig, wenn dein Repo **nicht** nur `web` enthält, sondern der Ordner tiefer liegt; dann `web` eintragen.
5. **Deploy** klicken und warten.
6. Wenn grün: **Visit** / die angezeigte **Production URL** öffnen → Login `planer@local` + Passwort.

Teil D ist fertig, sobald die Seite lädt und du dich einloggen kannst.

Was danach?

Situation	Was tun
Nur Code geändert	Änderungen committen & pushen → Vercel baut automatisch neu.
Datenbank-Schema geändert (Prisma)	Lokal: <code>export DATABASE_URL="..."</code> (Neon) → <code>npx prisma db push</code> → dann pushen & warten auf Deploy.
Neue Feiertage / Seed erneut	Lokal mit Neon- <code>DATABASE_URL</code> : <code>npx prisma db seed</code> (vorsicht: kann bestehende Seed-Daten überschreiben — bei euch meist unkritisch).
Passwort ändern	In Vercel <code>PLANNER_PASSWORD</code> anpassen + Redeploy ; lokal Seed ggf. mit neuem Passwort erneut (User-Update im Seed) oder Passwort in DB-Tool ändern.

Kurz-Checkliste

- ☐ Neon-Projekt + Connection String
- ☐ Lokal `DATABASE_URL`, `SESSION_SECRET`, `PLANNER_PASSWORD` → `db push` + `db seed`
- ☐ GitHub-Repo + Push aus Ordner `web`
- ☐ Vercel: Projekt importieren + **drei** Umgebungsvariablen + Deploy
- ☐ URL testen: `planer@local` + Passwort

Bei Problemen: Fehlermeldung aus Terminal oder Vercel **Build Logs** kopieren und gezielt nachfragen (ohne Connection-Strings zu teilen).

Ausführlichere technische Notizen: siehe `DEPLOYMENT.md` im gleichen Ordner.